5.8 Chebyshev Approximation

The Chebyshev polynomial of degree n is denoted $T_n(x)$ and is given by the explicit formula

$$T_n(x) = \cos(n \arccos x) \tag{5.8.1}$$

This may look trigonometric at first glance (and there is in fact a close relation between the Chebyshev polynomials and the discrete Fourier transform); however, (5.8.1) can be combined with trigonometric identities to yield explicit expressions for $T_n(x)$ (see Figure 5.8.1):

$$\begin{aligned} T_0(x) &= 1 \\ T_1(x) &= x \\ T_2(x) &= 2x^2 - 1 \\ T_3(x) &= 4x^3 - 3x \\ T_4(x) &= 8x^4 - 8x^2 + 1 \\ &\cdots \\ T_{n+1}(x) &= 2xT_n(x) - T_{n-1}(x) \quad n \geq 1. \end{aligned} \tag{5.8.2}$$

(There also exist inverse formulas for the powers of x in terms of the T_n's — see, e.g., [1].)

The Chebyshev polynomials are orthogonal in the interval $[-1, 1]$ over a weight $(1 - x^2)^{-1/2}$. In particular,

$$\int_{-1}^{1} \frac{T_i(x)T_j(x)}{\sqrt{1-x^2}}dx = \begin{cases} 0 & i \neq j \\ \pi/2 & i = j \neq 0 \\ \pi & i = j = 0 \end{cases} \tag{5.8.3}$$

The polynomial $T_n(x)$ has n zeros in the interval $[-1, 1]$, and they are located at the points

$$x = \cos\left(\frac{\pi(k + \frac{1}{2})}{n}\right) \qquad k = 0, 1, \ldots, n-1 \tag{5.8.4}$$

In this same interval there are $n + 1$ extrema (maxima and minima), located at

$$x = \cos\left(\frac{\pi k}{n}\right) \qquad k = 0, 1, \ldots, n \tag{5.8.5}$$

At all of the maxima $T_n(x) = 1$, while at all of the minima $T_n(x) = -1$; it is precisely this property that makes the Chebyshev polynomials so useful in polynomial approximation of functions.

The Chebyshev polynomials satisfy a discrete orthogonality relation as well as the continuous one (5.8.3): If $x_k (k = 0, \ldots, m-1)$ are the m zeros of $T_m(x)$ given by (5.8.4), and if $i, j < m$, then

$$\sum_{k=0}^{m-1} T_i(x_k)T_j(x_k) = \begin{cases} 0 & i \neq j \\ m/2 & i = j \neq 0 \\ m & i = j = 0 \end{cases} \tag{5.8.6}$$

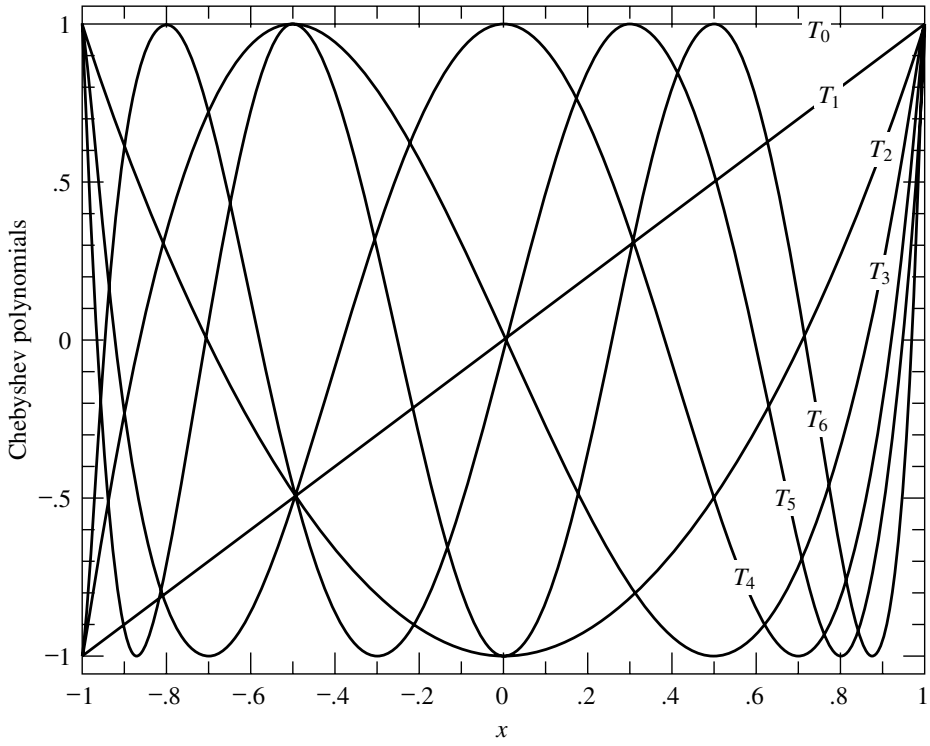

Figure 5.8.1. Chebyshev polynomials $T_0(x)$ through $T_6(x)$. Note that T_j has j roots in the interval $(-1, 1)$ and that all the polynomials are bounded between ± 1.

It is not too difficult to combine equations (5.8.1), (5.8.4), and (5.8.6) to prove the following theorem: If $f(x)$ is an arbitrary function in the interval $[-1, 1]$, and if N coefficients c_j, $j = 0, \ldots, N-1$, are defined by

$$\begin{aligned} c_j &= \frac{2}{N} \sum_{k=0}^{N-1} f(x_k) T_j(x_k) \\ &= \frac{2}{N} \sum_{k=0}^{N-1} f\left[\cos\left(\frac{\pi(k+\frac{1}{2})}{N}\right)\right] \cos\left(\frac{\pi j(k+\frac{1}{2})}{N}\right) \end{aligned} \tag{5.8.7}$$

then the approximation formula

$$f(x) \approx \left[\sum_{k=0}^{N-1} c_k T_k(x)\right] - \frac{1}{2} c_0 \tag{5.8.8}$$

is *exact* for x equal to all of the N zeros of $T_N(x)$.

For a fixed N, equation (5.8.8) is a polynomial in x that approximates the function $f(x)$ in the interval $[-1, 1]$ (where all the zeros of $T_N(x)$ are located). Why is this particular approximating polynomial better than any other one, exact on some other set of N points? The answer is *not* that (5.8.8) is necessarily more accurate

than some other approximating polynomial of the same order N (for some specified definition of "accurate"), but rather that (5.8.8) can be truncated to a polynomial of *lower* degree $m \ll N$ in a very graceful way, one that *does* yield the "most accurate" approximation of degree m (in a sense that can be made precise). Suppose N is so large that (5.8.8) is virtually a perfect approximation of $f(x)$. Now consider the truncated approximation

$$f(x) \approx \left[\sum_{k=0}^{m-1} c_k T_k(x) \right] - \frac{1}{2} c_0 \tag{5.8.9}$$

with the same c_j's, computed from (5.8.7). Since the $T_k(x)$'s are all bounded between ± 1, the difference between (5.8.9) and (5.8.8) can be no larger than the sum of the neglected c_k's ($k = m, \ldots, N-1$). In fact, if the c_k's are rapidly decreasing (which is the typical case), then the error is dominated by $c_m T_m(x)$, an oscillatory function with $m+1$ equal extrema distributed smoothly over the interval $[-1, 1]$. This smooth spreading out of the error is a very important property: The Chebyshev approximation (5.8.9) is very nearly the same polynomial as that holy grail of approximating polynomials the *minimax polynomial*, which (among all polynomials of the same degree) has the smallest maximum deviation from the true function $f(x)$. The minimax polynomial is very difficult to find; the Chebyshev approximating polynomial is almost identical and is very easy to compute!

So, given some (perhaps difficult) means of computing the function $f(x)$, we now need algorithms for implementing (5.8.7) and (after inspection of the resulting c_k's and choice of a truncating value m) evaluating (5.8.9). The latter equation then becomes an easy way of computing $f(x)$ for all subsequent time.

The first of these tasks is straightforward. A generalization of equation (5.8.7) that is here implemented is to allow the range of approximation to be between two arbitrary limits a and b, instead of just -1 to 1. This is effected by a change of variable

$$y \equiv \frac{x - \frac{1}{2}(b+a)}{\frac{1}{2}(b-a)} \tag{5.8.10}$$

and by the approximation of $f(x)$ by a Chebyshev polynomial in y.

It will be convenient for us to group a number of functions related to Chebyshev polynomials into a single object, even though discussion of their specifics is spread out over §5.8 – §5.11:

chebyshev.h

```
struct Chebyshev {
```
Object for Chebyshev approximation and related methods.

```
    Int n,m;
```
Number of total, and truncated, coefficients.

```
    VecDoub c;
    Doub a,b;
```
Approximation interval.

```
    Chebyshev(Doub func(Doub), Doub aa, Doub bb, Int nn);
```
Constructor. Approximate the function `func` in the interval [aa,bb] with `nn` terms.

```
    Chebyshev(VecDoub &cc, Doub aa, Doub bb)
        : n(cc.size()), m(n), c(cc), a(aa), b(bb) {}
```
Constructor from previously computed coefficients.

```
    Int setm(Doub thresh) {while (m>1 && abs(c[m-1])<thresh) m--; return m;}
```
Set `m`, the number of coefficients after truncating to an error level `thresh`, and return the value set.

```
    Doub eval(Doub x, Int m);
    inline Doub operator() (Doub x) {return eval(x,m);}
    Return a value for the Chebyshev fit, either using the stored m or else overriding it.

    Chebyshev derivative();          See §5.9.
    Chebyshev integral();

    VecDoub polycofs(Int m);         See §5.10.
    inline VecDoub polycofs() {return polycofs(m);}
    Chebyshev(VecDoub &pc);          See §5.11.

};
```

The first constructor, the one with an arbitrary function `func` as its first argument, calculates and saves `nn` Chebyshev coefficients that approximate `func` in the range `aa` to `bb`. (You can ignore for now the second constructor, which simply makes a `Chebyshev` object from already-calculated data.) Let us also note the method `setm`, which provides a quick way to truncate the Chebyshev series by (in effect) deleting, from the right, all coefficients smaller in magnitude than some threshold `thresh`.

chebyshev.h

```
Chebyshev::Chebyshev(Doub func(Doub), Doub aa, Doub bb, Int nn=50)
    : n(nn), m(nn), c(n), a(aa), b(bb)
```

Chebyshev fit: Given a function `func`, lower and upper limits of the interval [a,b], compute and save `nn` coefficients of the Chebyshev approximation such that $\texttt{func}(x) \approx [\sum_{k=0}^{\texttt{nn}-1} c_k T_k(y)] - c_0/2$, where y and x are related by (5.8.10). This routine is intended to be called with moderately large `n` (e.g., 30 or 50), the array of `c`'s subsequently to be truncated at the smaller value m such that c_m and subsequent elements are negligible.

```
{
    const Doub pi=3.141592653589793;
    Int k,j;
    Doub fac,bpa,bma,y,sum;
    VecDoub f(n);
    bma=0.5*(b-a);
    bpa=0.5*(b+a);
    for (k=0;k<n;k++) {                  We evaluate the function at the n points required
        y=cos(pi*(k+0.5)/n);                 by (5.8.7).
        f[k]=func(y*bma+bpa);
    }
    fac=2.0/n;
    for (j=0;j<n;j++) {                  Now evaluate (5.8.7).
        sum=0.0;
        for (k=0;k<n;k++)
            sum += f[k]*cos(pi*j*(k+0.5)/n);
        c[j]=fac*sum;
    }
}
```

If you find that the constructor's execution time is dominated by the calculation of N^2 cosines, rather than by the N evaluations of your function, then you should look ahead to §12.3, especially equation (12.4.16), which shows how fast cosine transform methods can be used to evaluate equation (5.8.7).

Now that we have the Chebyshev coefficients, how do we evaluate the approximation? One could use the recurrence relation of equation (5.8.2) to generate values for $T_k(x)$ from $T_0 = 1, T_1 = x$, while also accumulating the sum of (5.8.9). It is better to use Clenshaw's recurrence formula (§5.4), effecting the two processes simultaneously. Applied to the Chebyshev series (5.8.9), the recurrence is

$$
\begin{aligned}
d_{m+1} &\equiv d_m \equiv 0 \\
d_j &= 2xd_{j+1} - d_{j+2} + c_j \qquad j = m-1, m-2, \ldots, 1 \\
f(x) &\equiv d_0 = xd_1 - d_2 + \tfrac{1}{2}c_0
\end{aligned}
\tag{5.8.11}
$$

chebyshev.h

```
Doub Chebyshev::eval(Doub x, Int m)
```

Chebyshev evaluation: The Chebyshev polynomial $\sum_{k=0}^{m-1} c_k T_k(y) - c_0/2$ is evaluated at a point $y = [x - (b+a)/2]/[(b-a)/2]$, and the result is returned as the function value.

```
{
    Doub d=0.0,dd=0.0,sv,y,y2;
    Int j;
    if ((x-a)*(x-b) > 0.0) throw("x not in range in Chebyshev::eval");
    y2=2.0*(y=(2.0*x-a-b)/(b-a));          Change of variable.
    for (j=m-1;j>0;j--) {                  Clenshaw's recurrence.
        sv=d;
        d=y2*d-dd+c[j];
        dd=sv;
    }
    return y*d-dd+0.5*c[0];                Last step is different.
}
```

The method `eval` has an argument for specifying how many leading coefficients `m` should be used in the evaluation. If you simply want to use a stored value of `m` that was set by a previous call to `setm` (or, by hand, by you), then you can use the `Chebyshev` object as a functor. For example,

```
Chebyshev approxfunc(func,0.,1.,50);
approxfunc.setm(1.e-8);
...
y = approxfunc(x);
```

If we are approximating an *even* function on the interval $[-1, 1]$, its expansion will involve only even Chebyshev polynomials. It is wasteful to construct a `Chebyshev` object with all the odd coefficients zero [2]. Instead, using the half-angle identity for the cosine in equation (5.8.1), we get the relation

$$
T_{2n}(x) = T_n(2x^2 - 1) \tag{5.8.12}
$$

Thus we can construct a more efficient `Chebyshev` object for even functions simply by replacing the function's argument x by $2x^2 - 1$, and likewise when we evaluate the Chebyshev approximation.

An odd function will have an expansion involving only odd Chebyshev polynomials. It is best to rewrite it as an expansion for the function $f(x)/x$, which involves only even Chebyshev polynomials. This has the added benefit of giving accurate values for $f(x)/x$ near $x = 0$. Don't try to construct the series by evaluating $f(x)/x$ numerically, however. Rather, the coefficients c'_n for $f(x)/x$ can be found from those for $f(x)$ by recurrence:

$$
\begin{aligned}
c'_{N+1} &= 0 \\
c'_{n-1} &= 2c_n - c'_{n+1}, \qquad n = N-1, N-3, \ldots
\end{aligned}
\tag{5.8.13}
$$

Equation (5.8.13) follows from the recurrence relation in equation (5.8.2).

If you insist on evaluating an odd Chebyshev series, the efficient way is to once again to replace x by $y = 2x^2 - 1$ as the argument of your function. Now, however,

you must also change the last formula in equation (5.8.11) to be

$$f(x) = x[(2y - 1)d_1 - d_2 + c_0] \tag{5.8.14}$$

and change the corresponding line in `eval`.

5.8.1 Chebyshev and Exponential Convergence

Since first mentioning *truncation error* in §1.1, we have seen many examples of algorithms with an adjustable order, say M, such that the truncation error decreases as the Mth power of something. Examples include most of the interpolation methods in Chapter 3 and most of the quadrature methods in Chapter 4. In these examples there is also another parameter, N, which is the number of points at which a function will be evaluated.

We have many times warned that "higher order does not necessarily give higher accuracy." That remains good advice when N is held fixed while M is increased. However, a recently emerging theme in many areas of scientific computation is the use of methods that allow, in very special cases, M and N to be increased *together*, with the result that errors not only do decrease with higher order, but decrease exponentially!

The common thread in almost all of these relatively new methods is the remarkable fact that *infinitely smooth* functions become *exponentially* well determined by N sample points as N is increased. Thus, mere power-law convergence may be just a consequence of either (i) functions that are not smooth enough, or (ii) endpoint effects.

We already saw several examples of this in Chapter 4. In §4.1 we pointed out that high-order quadrature rules can have interior weights of unity, just like the trapezoidal rule; all of the "high-orderness" is obtained by a proper treatment near the boundaries. In §4.5 we further saw that variable transformations that push the boundaries off to infinity produce rapidly converging quadrature algorithms. In §4.5.1 we in fact proved exponential convergence, as a consequence of the Euler-Maclaurin formula. Then in §4.6 we remarked on the fact that the convergence of Gaussian quadratures could be exponentially rapid (an example, in the language above, of increasing M and N simultaneously).

Chebyshev approximation can be exponentially convergent for a different (though related) reason: Smooth *periodic* functions avoid endpoint effects by not having endpoints at all! Chebyshev approximation can be viewed as mapping the x interval $[-1, 1]$ onto the angular interval $[0, \pi]$ (cf. equations 5.8.4 and 5.8.5) in such a way that any infinitely smooth function on the interval $[-1, 1]$ becomes an infinitely smooth, even, periodic function on $[0, 2\pi]$. Figure 5.8.2 shows the idea geometrically. By projecting the abscissas onto a semicircle, a half-period is produced. The other half-period is obtained by reflection, or could be imagined as the result of projecting the function onto an identical lower semicircle. The zeros of the Chebyshev polynomial, or nodes of a Chebyshev approximation, are equally spaced on the circle, where the Chebyshev polynomial itself is a cosine function (cf. equation 5.8.1). This illustrates the close connection between Chebyshev approximation and periodic functions on the circle; in Chapter 12, we will apply the discrete Fourier transform to such functions in an almost equivalent way (§12.4.2).

The reason that Chebyshev works so well (and also why Gaussian quadratures work so well) is thus seen to be intimately related to the special way that the the

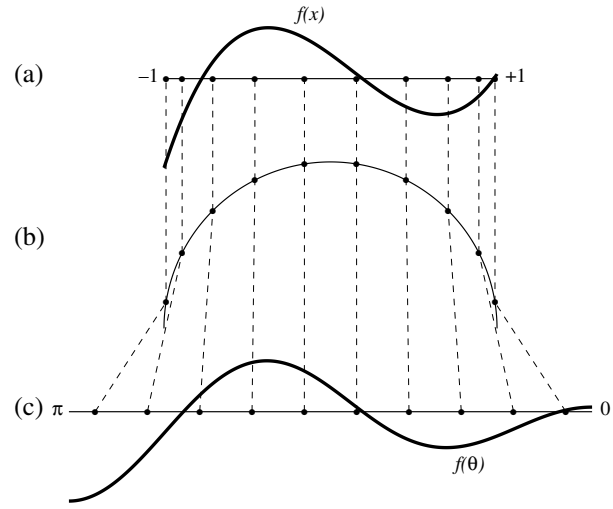

Figure 5.8.2. Geometrical construction showing how Chebyshev approximation is related to periodic functions. A smooth function on the interval is plotted in (a). In (b), the abscissas are mapped to a semicircle. In (c), the semicircle is unrolled. Because of the semicircle's vertical tangents, the function is now nearly constant at the endpoints. In fact, if reflected into the interval $[\pi, 2\pi]$, it is a smooth, even, periodic function on $[0, 2\pi]$.

sample points are bunched up near the endpoints of the interval. Any function that is bounded on the interval will have a convergent Chebyshev approximation as $N \rightarrow \infty$, even if there are nearby poles in the complex plane. For functions that are not infinitely smooth, the actual rate of convergence depends on the smoothness of the function: the more deriviatives that are bounded, the greater the convergence rate. For the special case of a C^∞ function, the convergence is exponential. In §3.0, in connection with polynomial interpolation, we mentioned the other side of the coin: equally spaced samples on the interval are about the *worst* possible geometry and often lead to ill-conditioned problems.

Use of the sampling theorem (§4.5, §6.9, §12.1, §13.11) is often closely associated with exponentially convergent methods. We will return to many of the concepts of exponentially convergent methods when we discuss spectral methods for partial differential equations in §20.7.

CITED REFERENCES AND FURTHER READING:

Arfken, G. 1970, *Mathematical Methods for Physicists*, 2nd ed. (New York: Academic Press), p. 631.[1]

Clenshaw, C.W. 1962, *Mathematical Tables*, vol. 5, National Physical Laboratory (London: H.M. Stationery Office).[2]

Goodwin, E.T. (ed.) 1961, *Modern Computing Methods*, 2nd ed. (New York: Philosophical Library), Chapter 8.

Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §4.4.1, p. 104.

Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), §6.5.2, p. 334.

Carnahan, B., Luther, H.A., and Wilkes, J.O. 1969, *Applied Numerical Methods* (New York: Wiley), §1.10, p. 39.